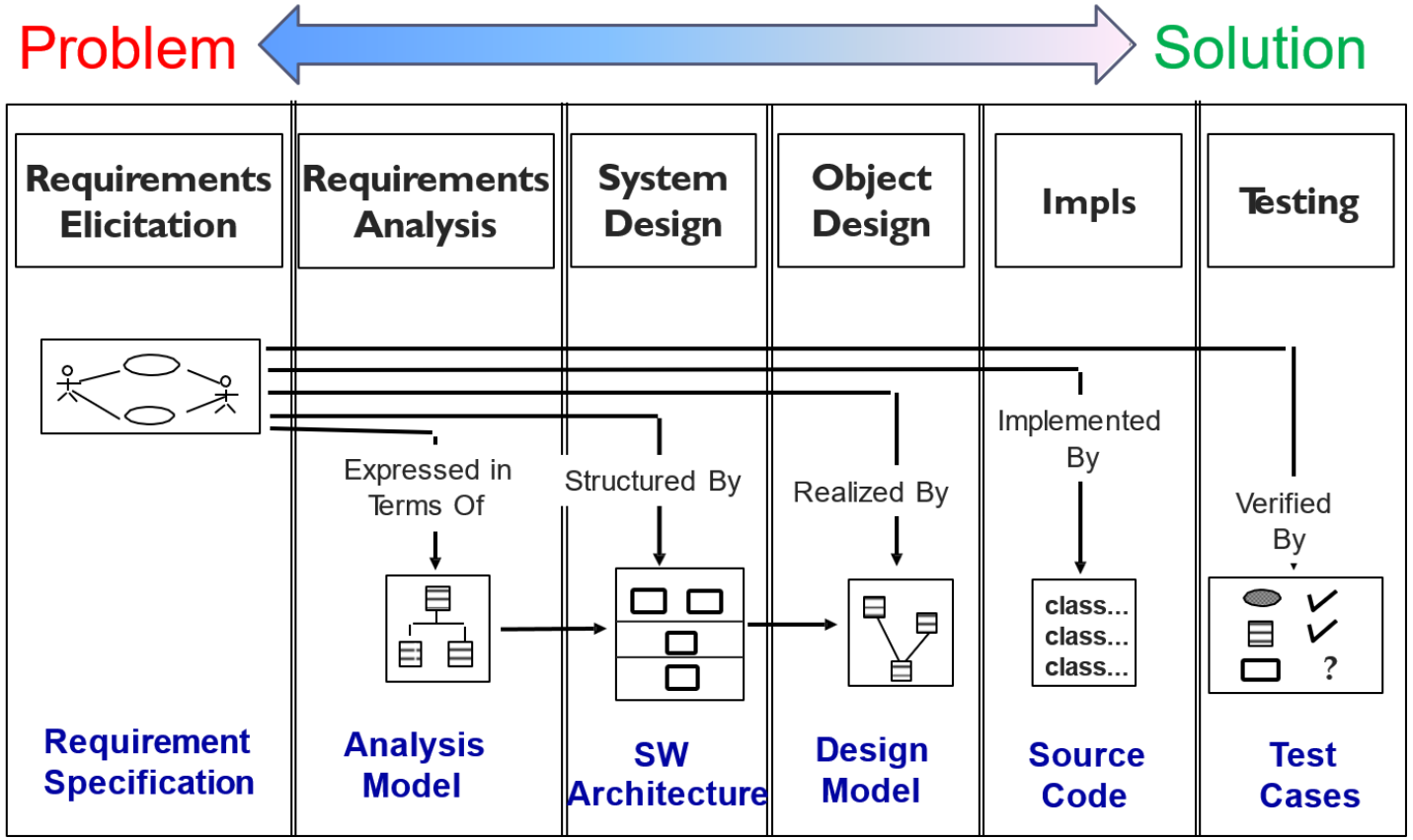
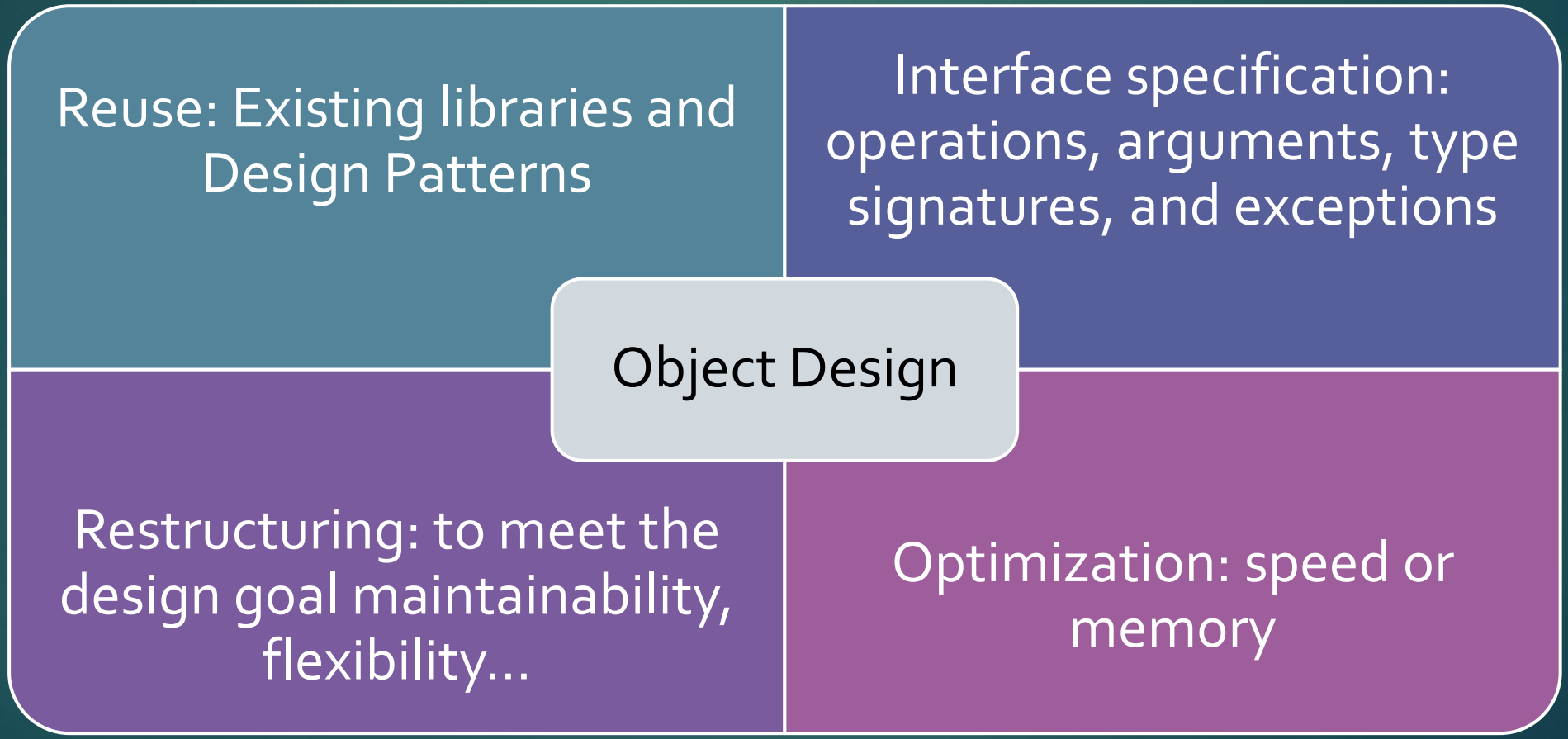
Lecture 13: Object Design

Liu Yang



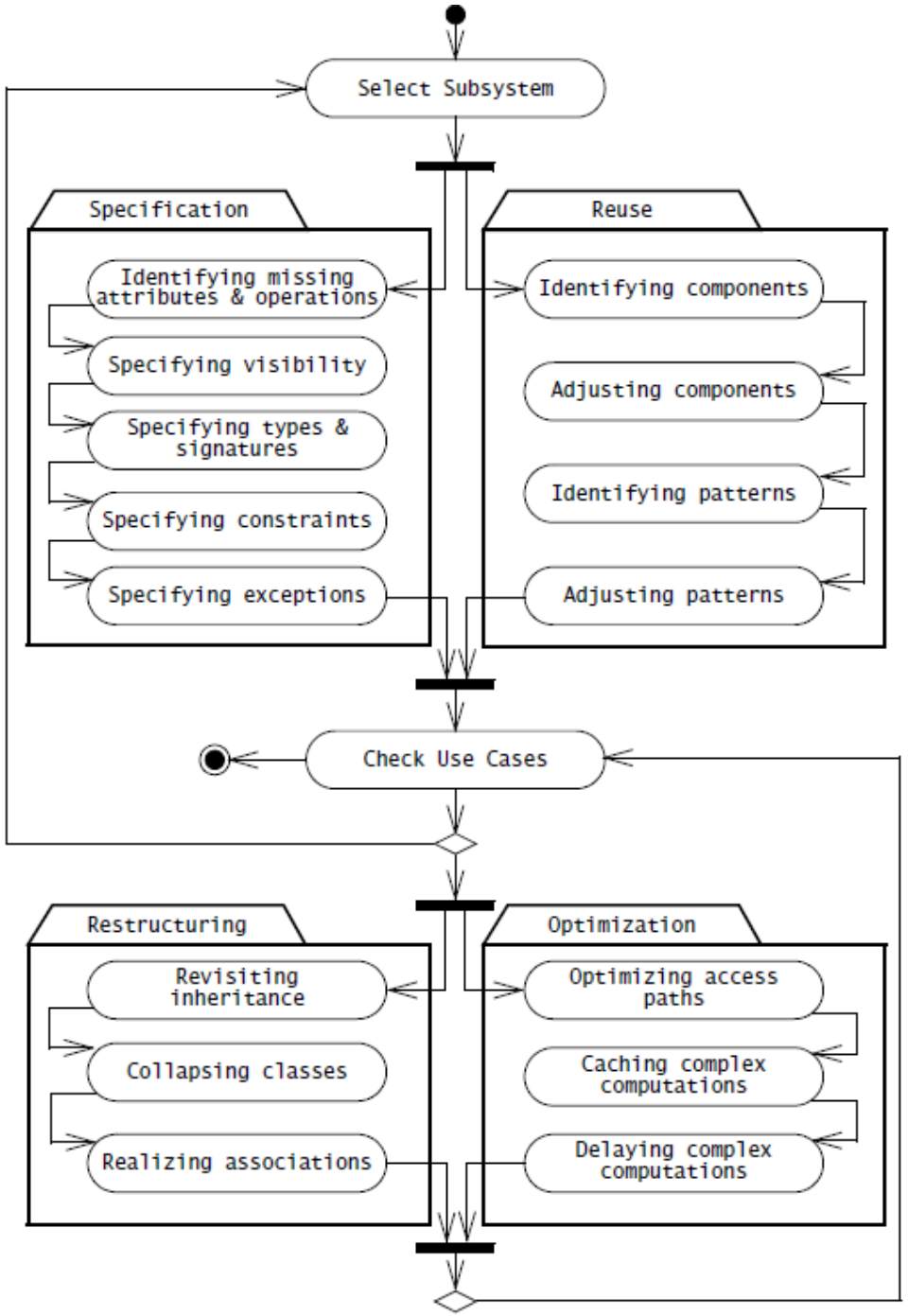
SDLC activities

Object Design

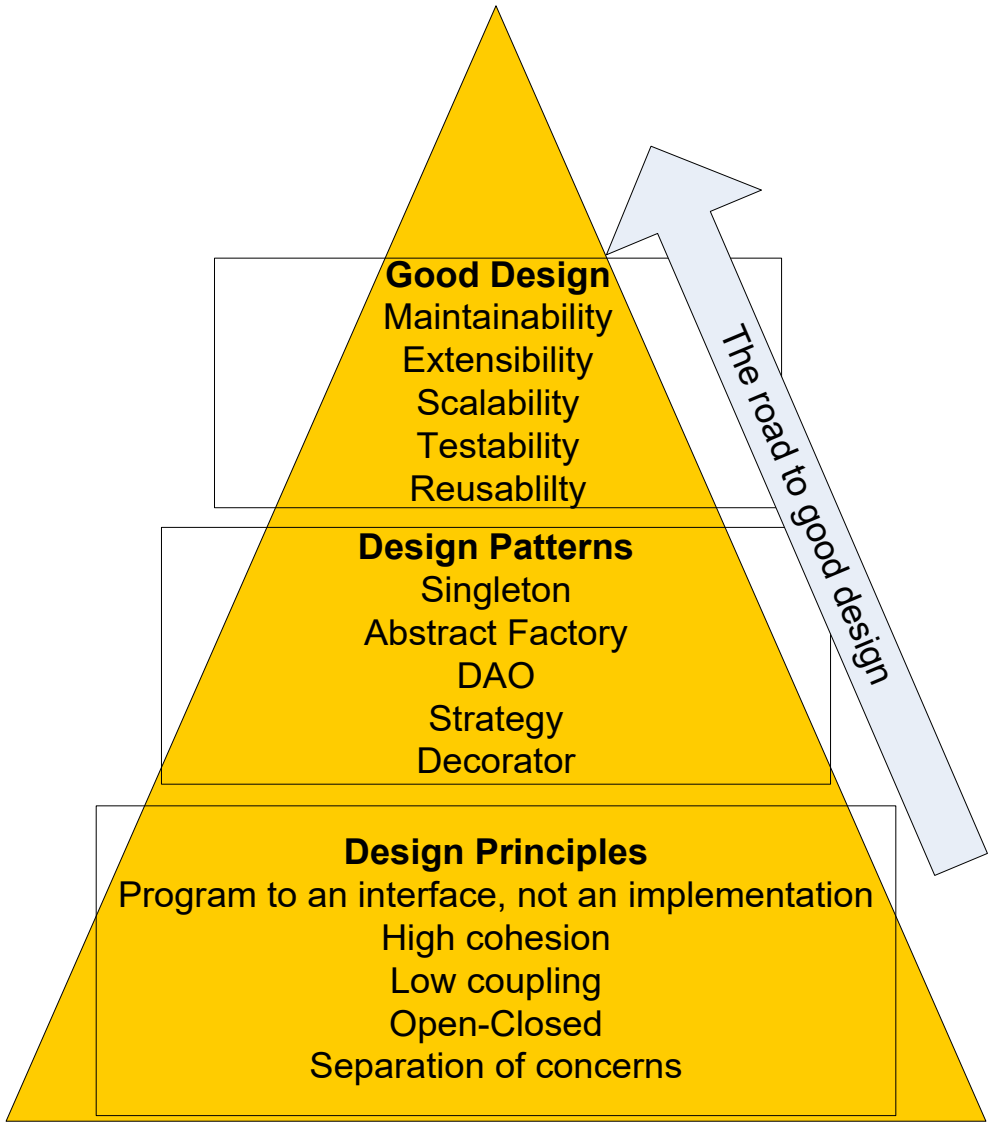


Object Design

(alternative view)

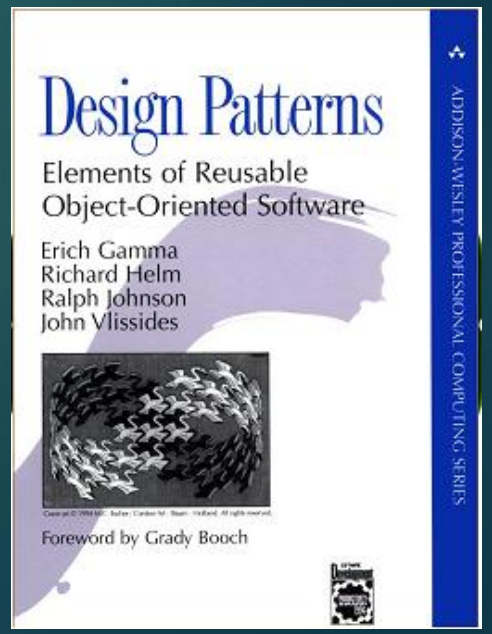


Why use Design Patterns?



What is a Design Pattern?

- A design pattern
  - a proven solution to a problem in a context.
  - abstracts a recurring design structure
  - A template with class and/or object
    - dependencies,
    - structures,
    - interactions, or
    - conventions



"Gang of Four" (GoF) Book, 1996

4 Elements of a Design Pattern



Name

Describes the pattern

Adds to common terminology for facilitating communication (i.e. not just sentence enhancers)



Problem

Describes when to apply the pattern

Answers - What is the pattern trying to solve?



Solution

Describes elements, relationships, responsibilities, and collaborations which make up the design



Consequences

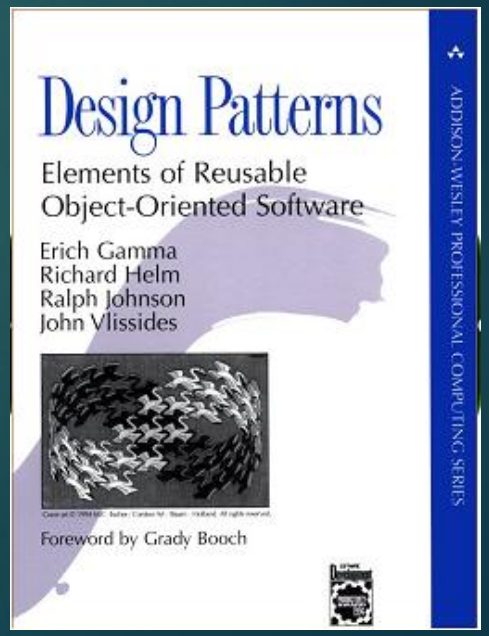
Results of applying the pattern

Benefits and Costs

Subjective depending on concrete scenarios

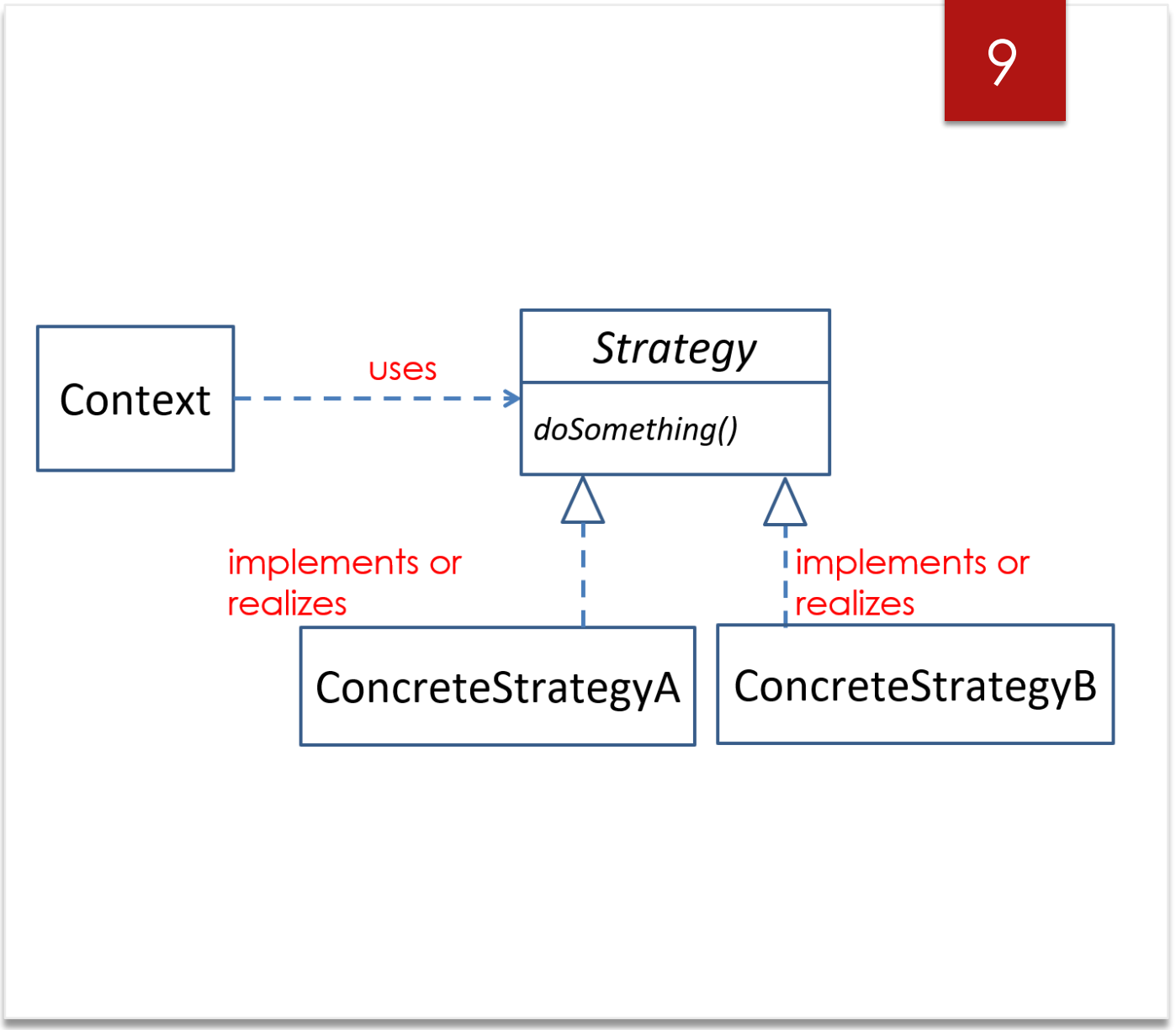
GoF Patterns (23)

- *Creational Patterns*
  - **Abstract Factory**
  - Builder
  - Factory Method
  - Prototype
  - Singleton
- *Structural Patterns*
  - Adapter
  - Bridge
  - Composite
  - Decorator
  - **Façade**
  - Flyweight
  - Proxy
- *Behavioral Patterns*
  - Chain of Responsibility
  - Command
  - Interpreter
  - Iterator
  - Mediator
  - Memento
  - **Observer**
  - State
  - **Strategy**
  - Template Method
  - Visitor



Strategy Pattern

- Design problem: A set of algorithms or objects should be interchangeable.
- Solution: Strategy Pattern

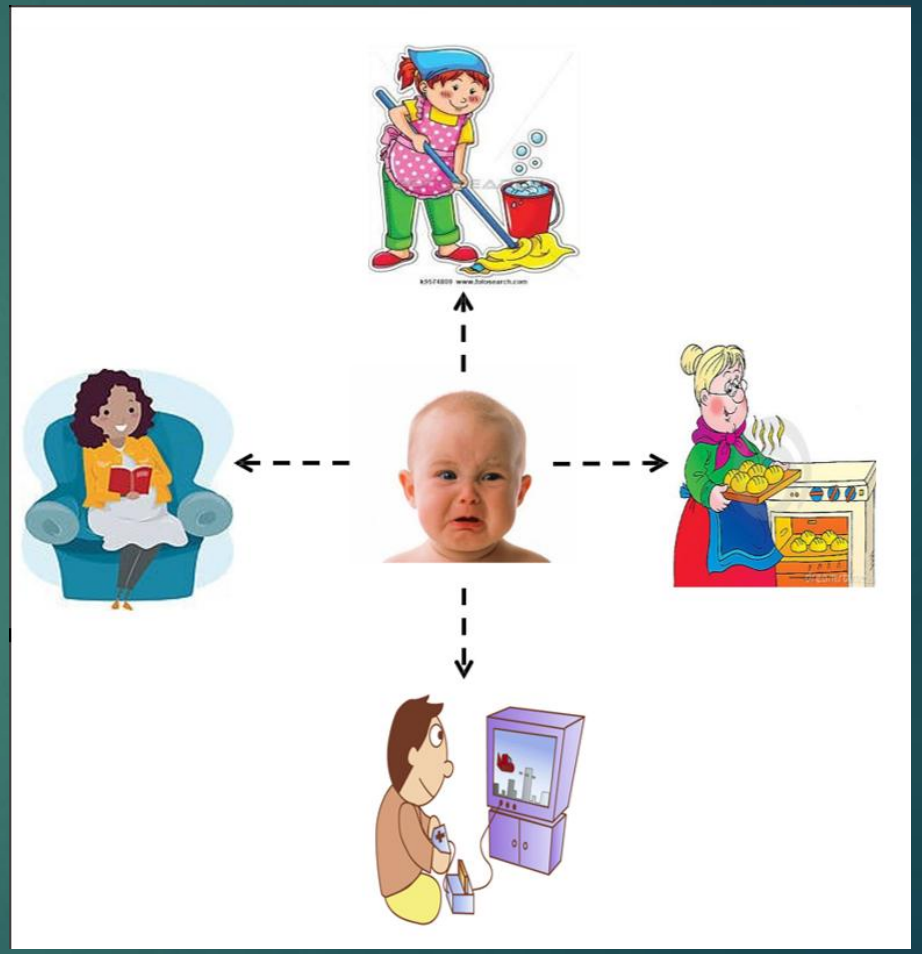


Strategy Pattern

- Pros
  - Easy extension for new strategies
  - Provides encapsulation, hides implementation
  - Allows behavior change at runtime
- Cons
  - Results in complex, hard to understand code if overused

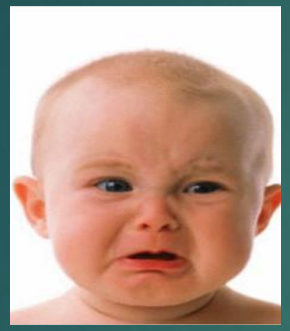
When to Use the Observer Pattern

- When you need many objects (called observers) to receive an update when another object (called subject) changes



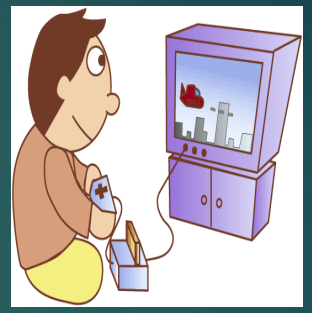
a.k.a. Subscriber – Publisher Pattern
Consumer – Producer Pattern

Analysis of Layered Structure Style



Subject

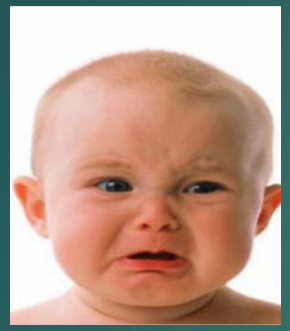
- I do not know (or care) who are looking after me and how many are looking after me
- I want to let whoever are looking after me know when I cry
- It is up to caregivers to decide what to do with me crying



Observer

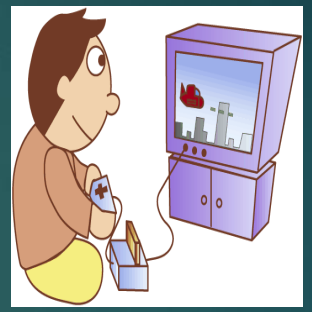
- Sometimes I want to look after the baby, sometimes I do not
- I want to know the incident once the baby cries, but I do not know when he will cry
- I cannot constantly check the baby's status because I have to do something else important

Analysis of Layered Structure Style



Subject

- I do not know (or care) who are looking after me and how many are looking after me
- I want to let whoever are looking after me know when I cry
- It is up to caregivers to decide what to do with me crying

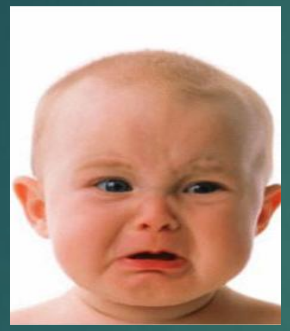


Observer

- Sometimes I want to look after the baby, sometimes I do not
- I want to know the incident once the baby cries, but I do not know when he will cry
- I cannot constantly check the baby's status because I have to do something else important

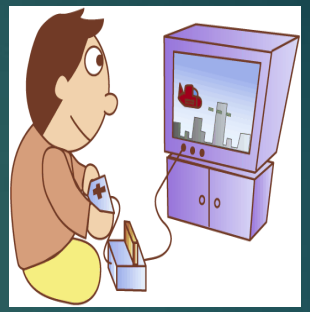
What kind of coupling (tight or loose) is preferred? Why?

Analysis of Layered Structure Style



Subject

- I do not know (or care) who are looking after me and how many are looking after me
- I want to let whoever are looking after me know when I cry
- It is up to caregivers to decide what to do with me crying



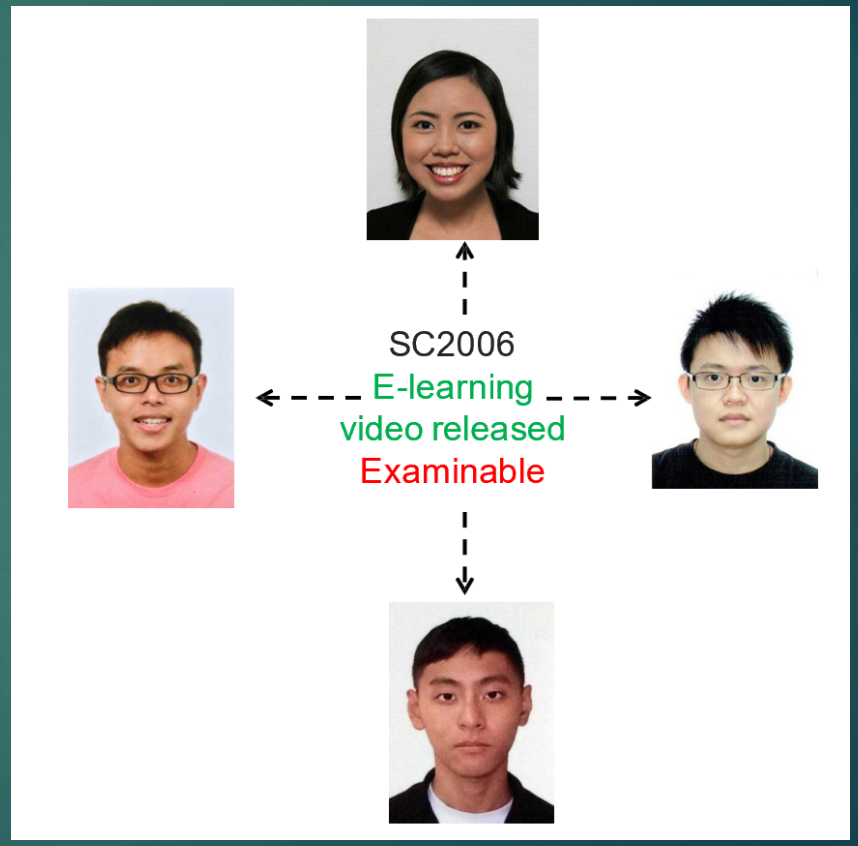
Observer

- Sometimes I want to look after the baby, sometimes I do not
- I want to know the incident once the baby cries, but I do not know when he will cry
- I cannot constantly check the baby's status because I have to do something else important

***Loose coupling** is a benefit for both sides!*

When to Use the Observer Pattern

- When you need many objects (called observers) to receive an update when another object (called subject) changes
  - *Who's the Subject?*
  - *Who's the Observer?*



Analysis of Layered Structure Style

NTULearn



Subject

- It does not matter who register the course and how many register the course
- I want to let whoever register the course know the latest announcement
- It is up to course takers to decide what to do with the change

Observer

- I want to be able to register/dropoff the course
- I want to know the latest announcement once it occurs, but I do not know when it will occur
- I cannot constantly check the announcement because I have to do something else important

Analysis of Layered Structure Style

NTULearn



Subject

- It does not matter who register the course and how many register the course
- I want to let whoever register the course know the latest announcement
- It is up to course takers to decide what to do with the change

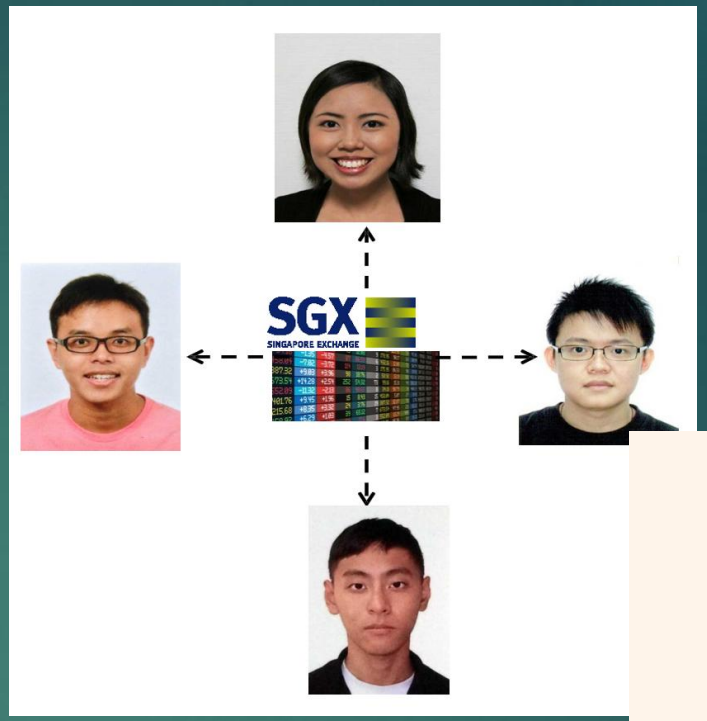
Observer

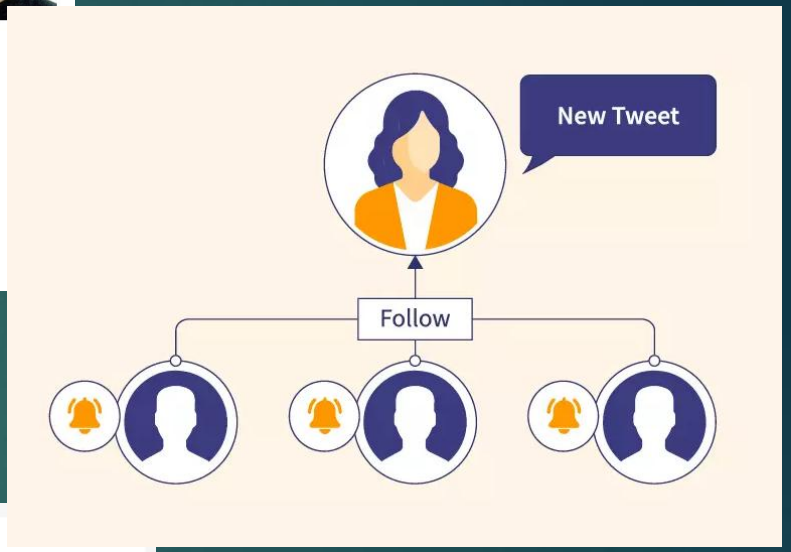
- I want to be able to register/dropoff the course
- I want to know the latest announcement once it occurs, but I do not know when it will occur
- I cannot constantly check the announcement because I have to do something else important

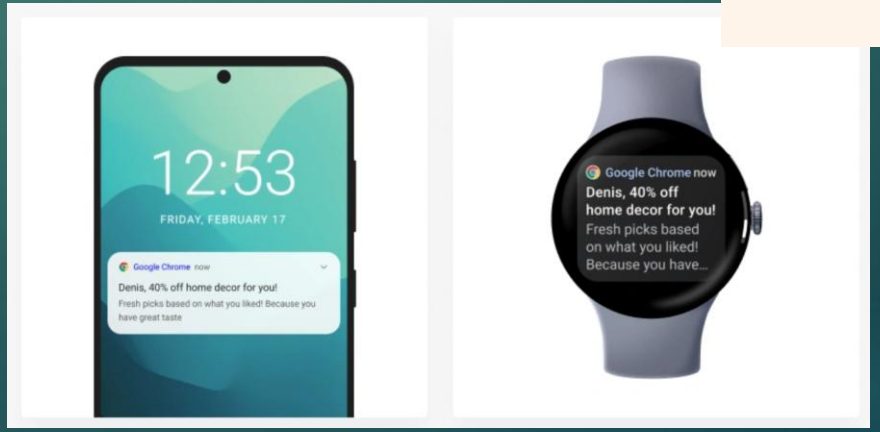
***Loose coupling** is a benefit for both sides!*

When to Use the Observer Pattern

- When you need many objects (called observers) to receive an update when another object (called subject) changes
  - *Who's the Subject?*
  - *Who's the Observer?*







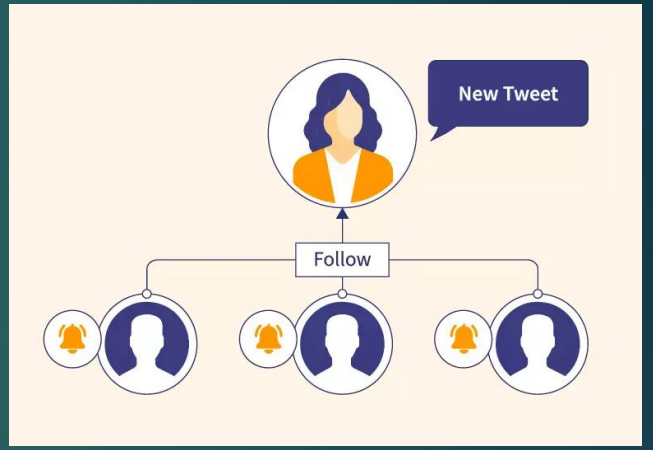
“Observer Pattern – Design Problems: *Easy Subscription and Automatic Notification*”



Defines a one-to-many dependency between objects so that when one object changes state, all of its dependents are notified and updated automatically.

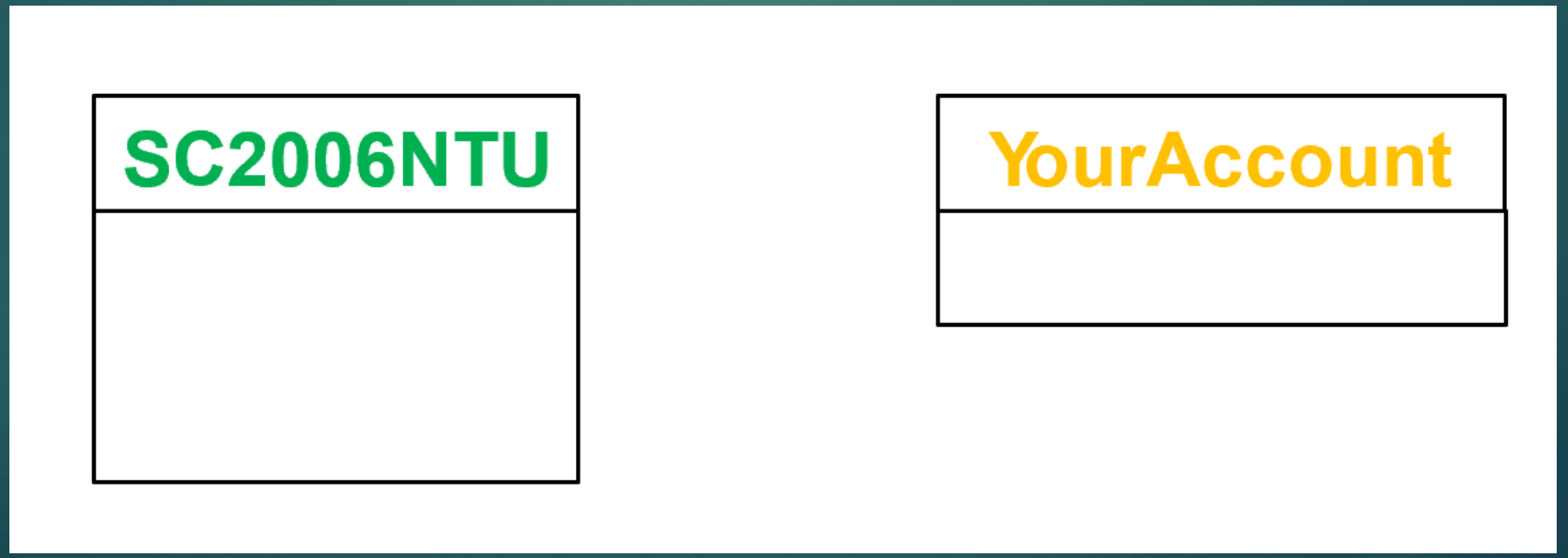
How to Resolve these Problems?

- Subscription mechanism
  - Observers freely register/unregister their interests in Subject
- Notification mechanism
  - Subject propagates the change to Observer when the change occurs



Understand How it Works: NTULearn Case Study

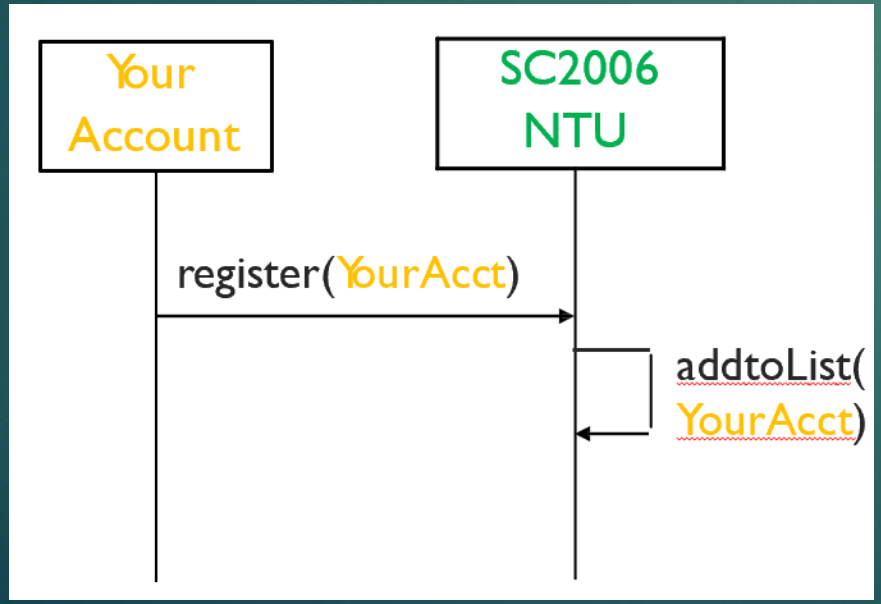
- In NTULearn system
  - **Subject**: SC2006NTU
  - **Observers**: Course takers (your NTU account)

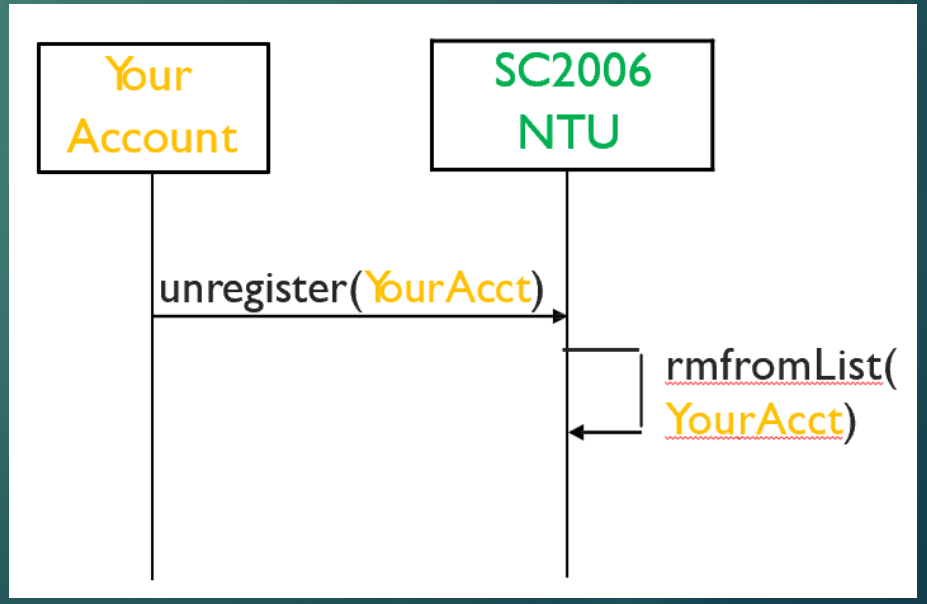


Subscription Mechanism

- **Add to course**
  (i.e., register)
- **Remove from course**
  (i.e., unregister)

SE2006NTU
register(YourAccount) unregister(YourAccount)

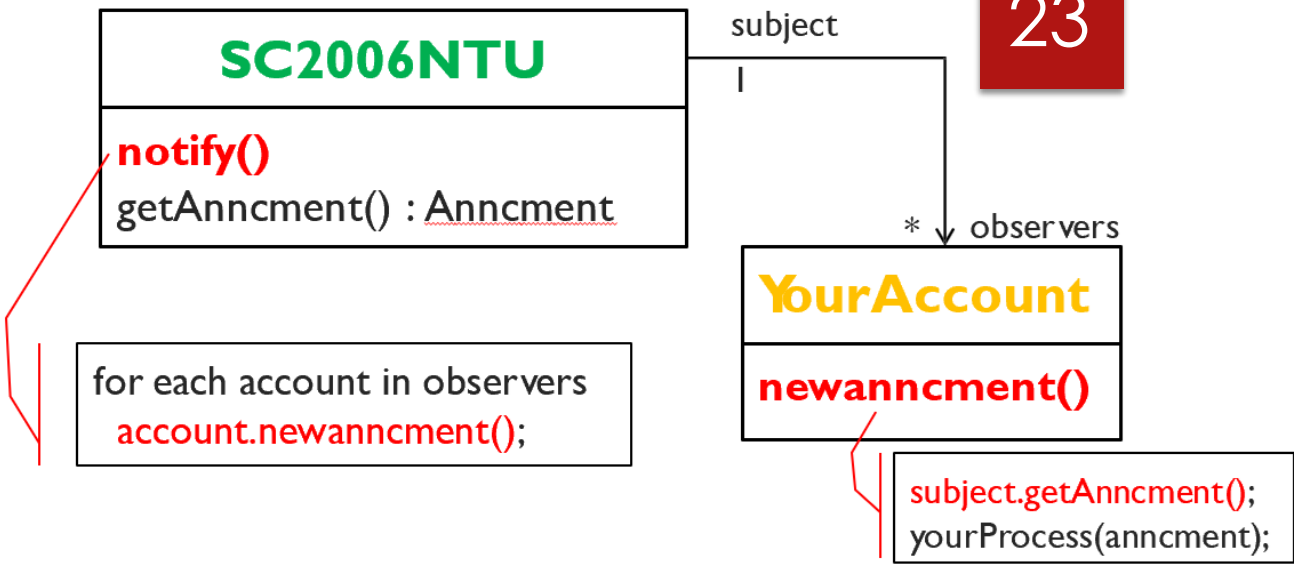


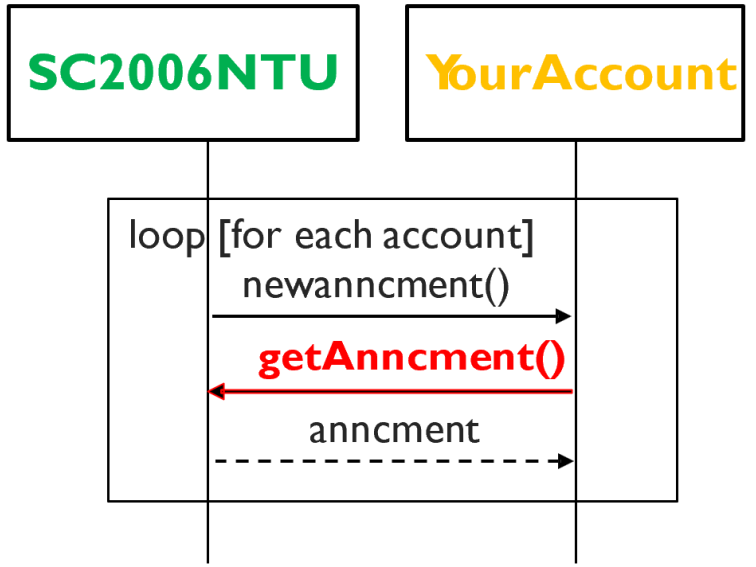


Notification Mechanism – Pull Update

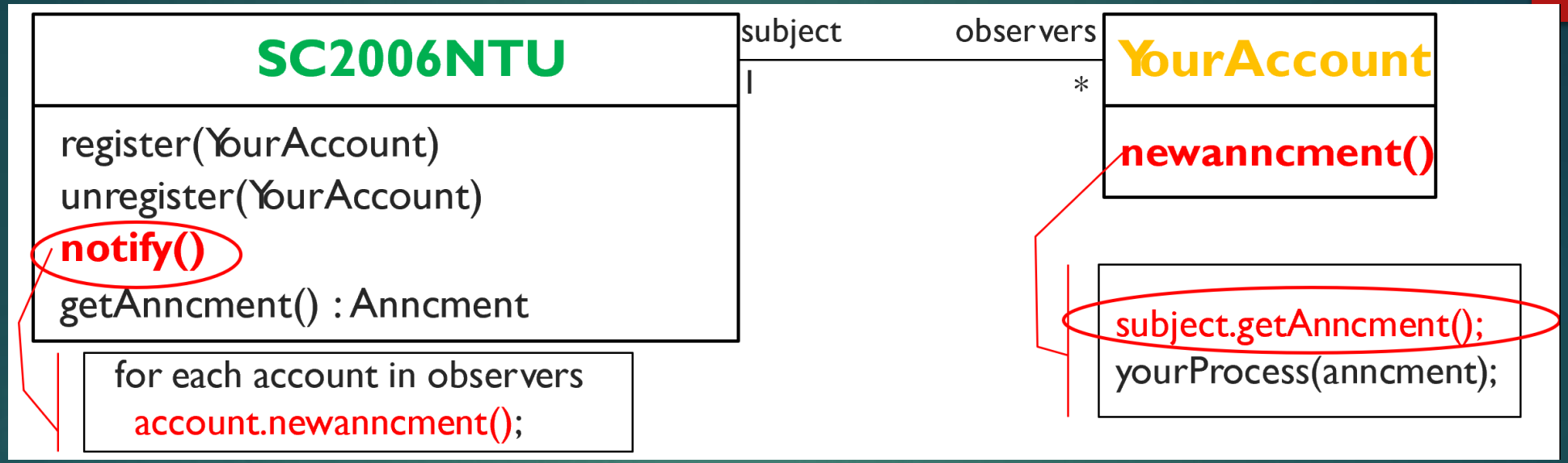
- **SC2006NTU** sends simple notification (e.g., new announcement)
- **YourAccount** calls back for details

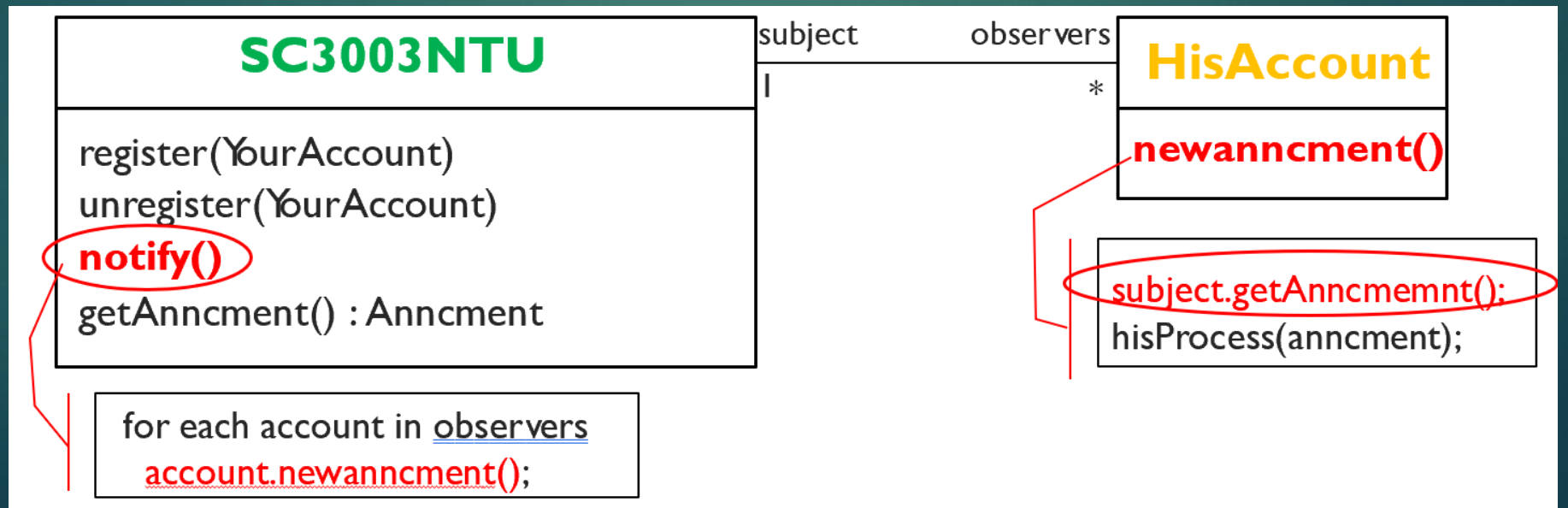




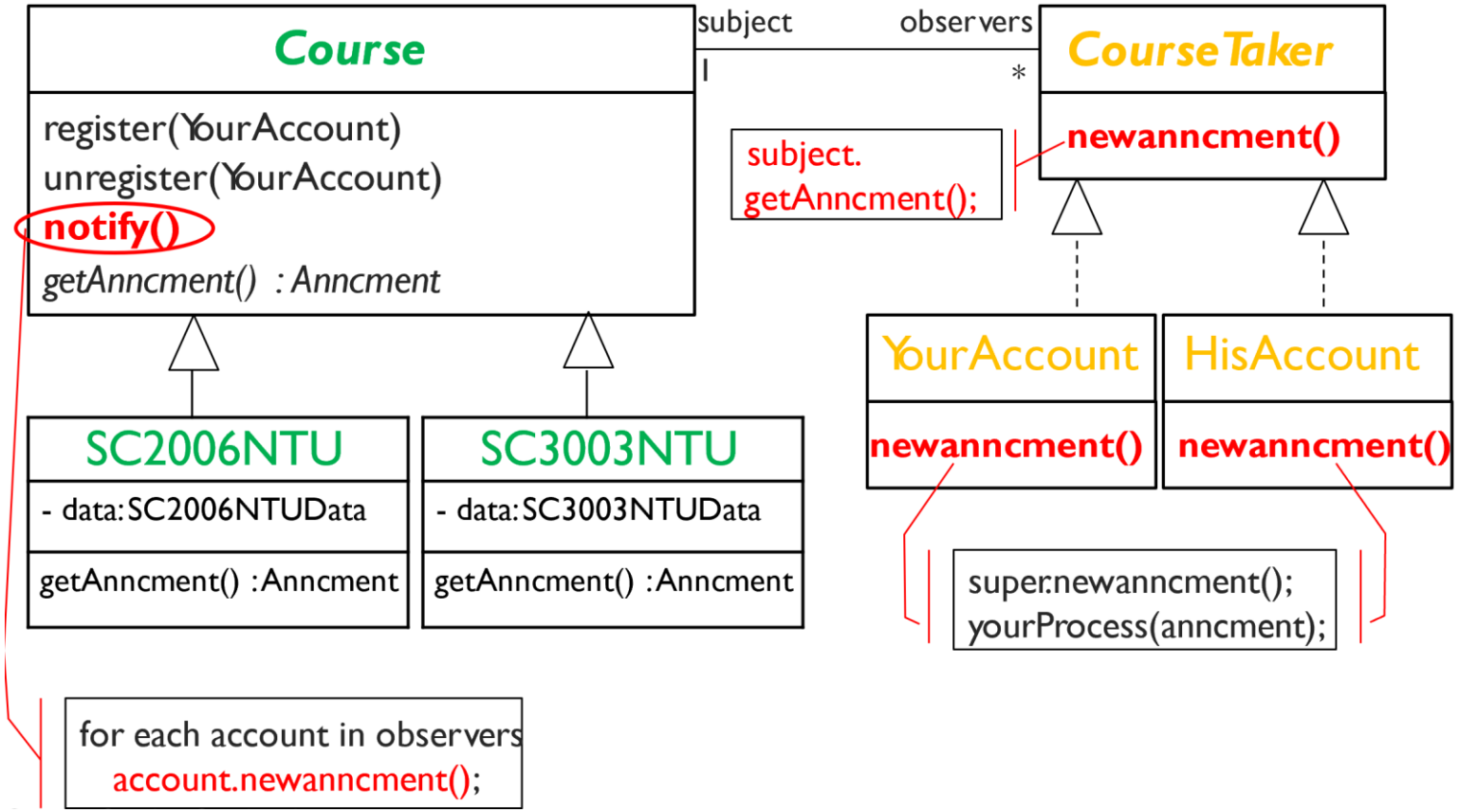


Pull it Together in a Class Diagram





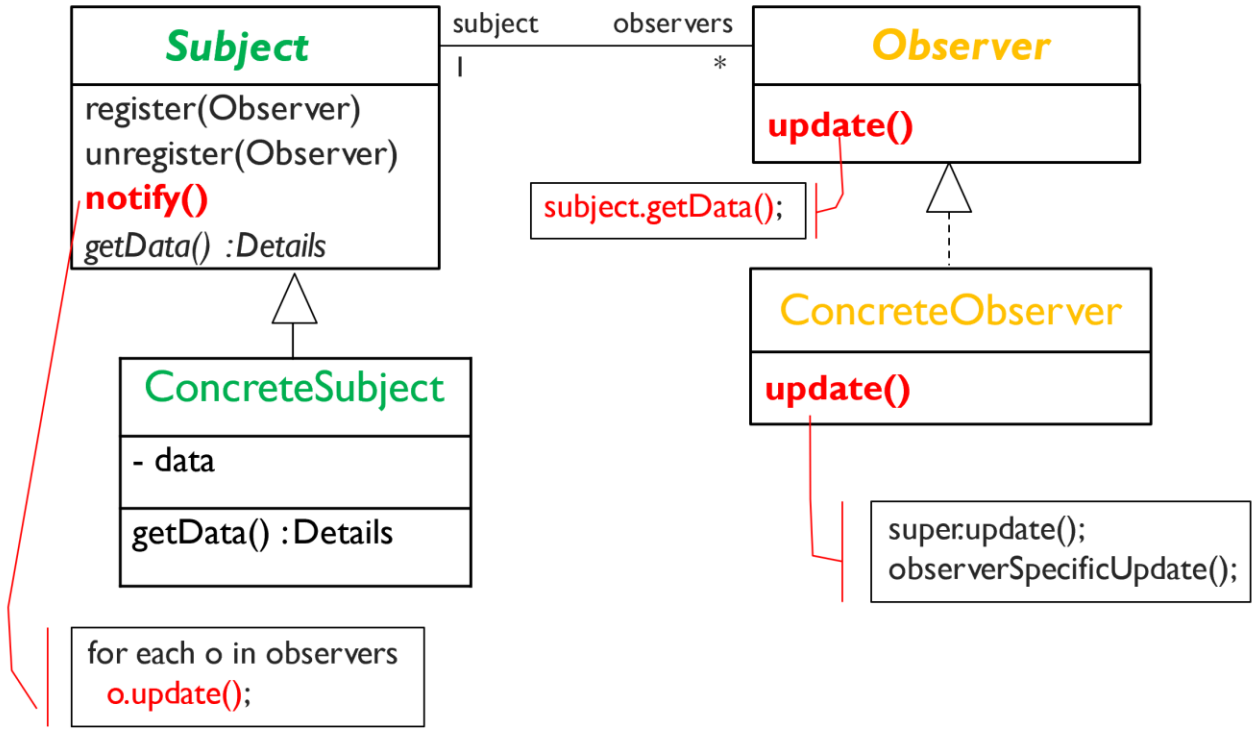
Abstract Commonalities into Superclasses – CourseTaker – Course Design



Observer Pattern Template

- **Subject** sends simple notification (e.g., new announcement)
- **Observer** calls back for details

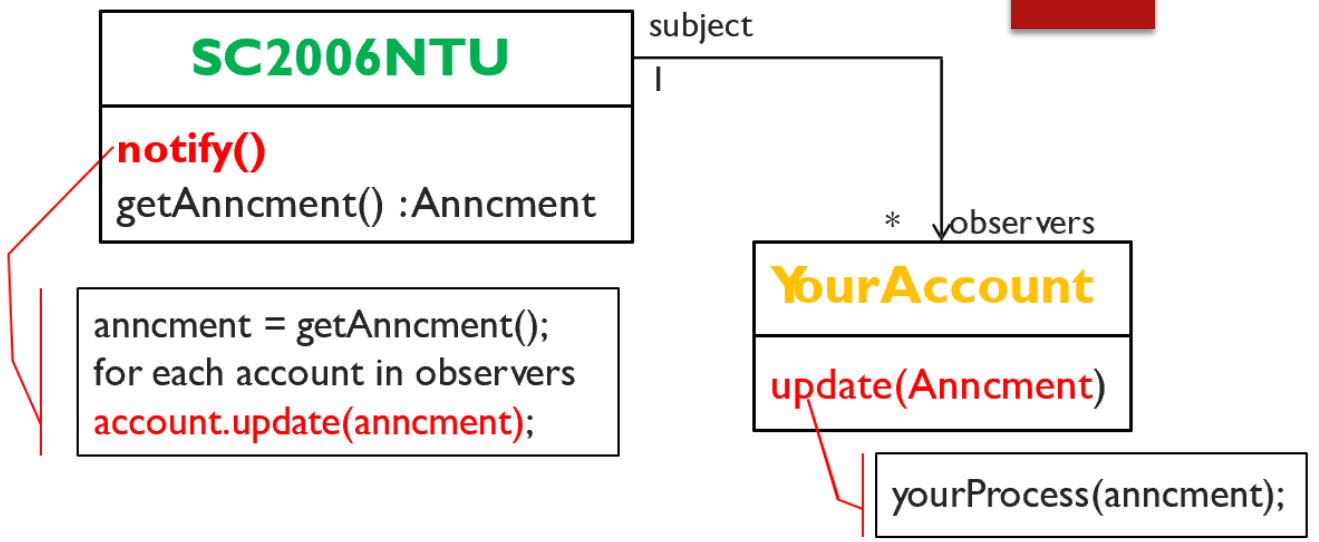


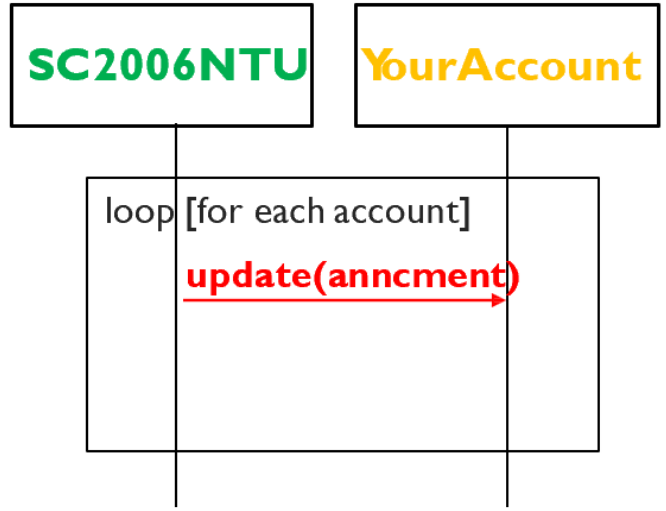


Notification Mechanism – *Push Update*

- **SC2006NTU** sends simple notification (e.g., new announcement)
- **YourAccount** does not need to call back



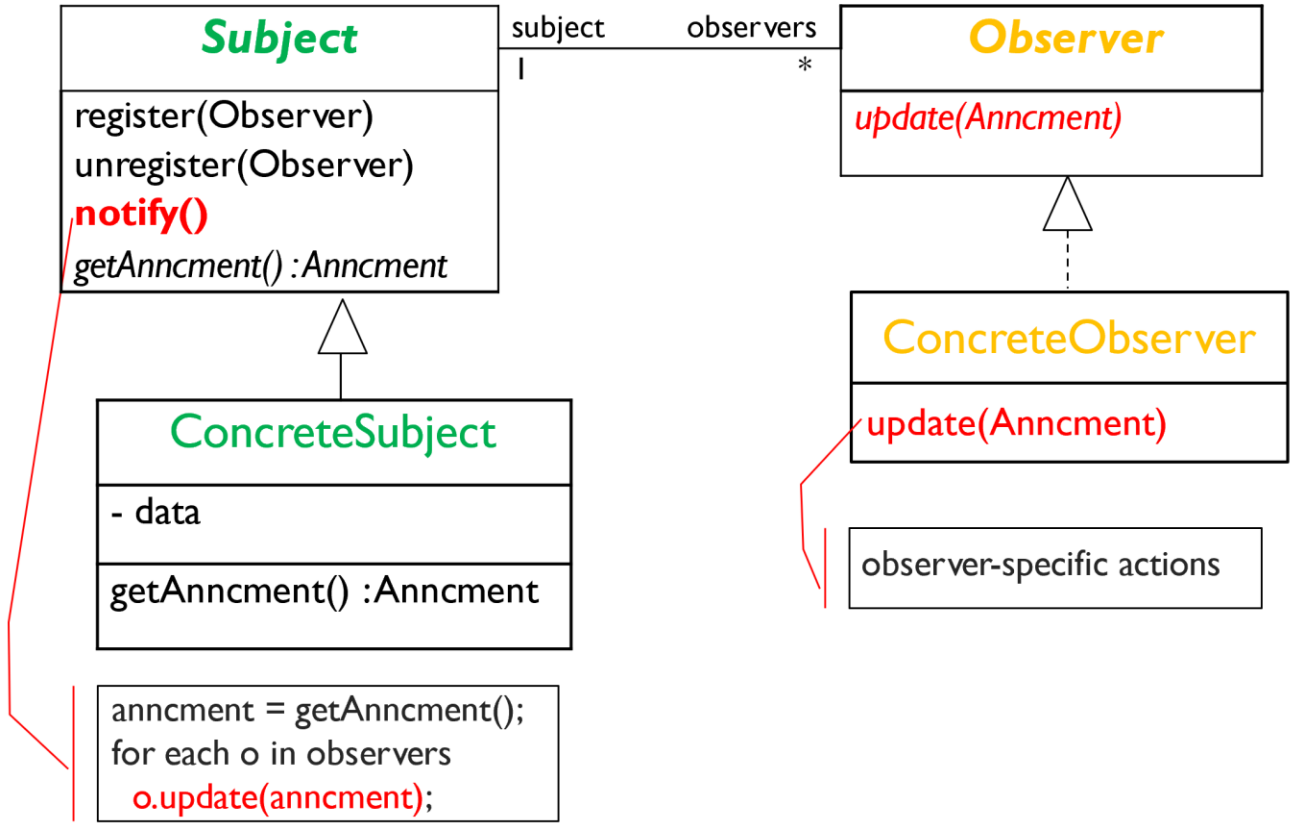




Observer Pattern Template

- **Subject** sends simple notification (e.g., new announcement)
- **Observer** does not need to call back





Update Protocol: Pull or Push?

Pull update

- Subject sends simple notification (via Observer.update()).
- The Observer calls back (via Subject.getData()) for details explicitly (if Observer is interested) – two way communication.

Which one is better?

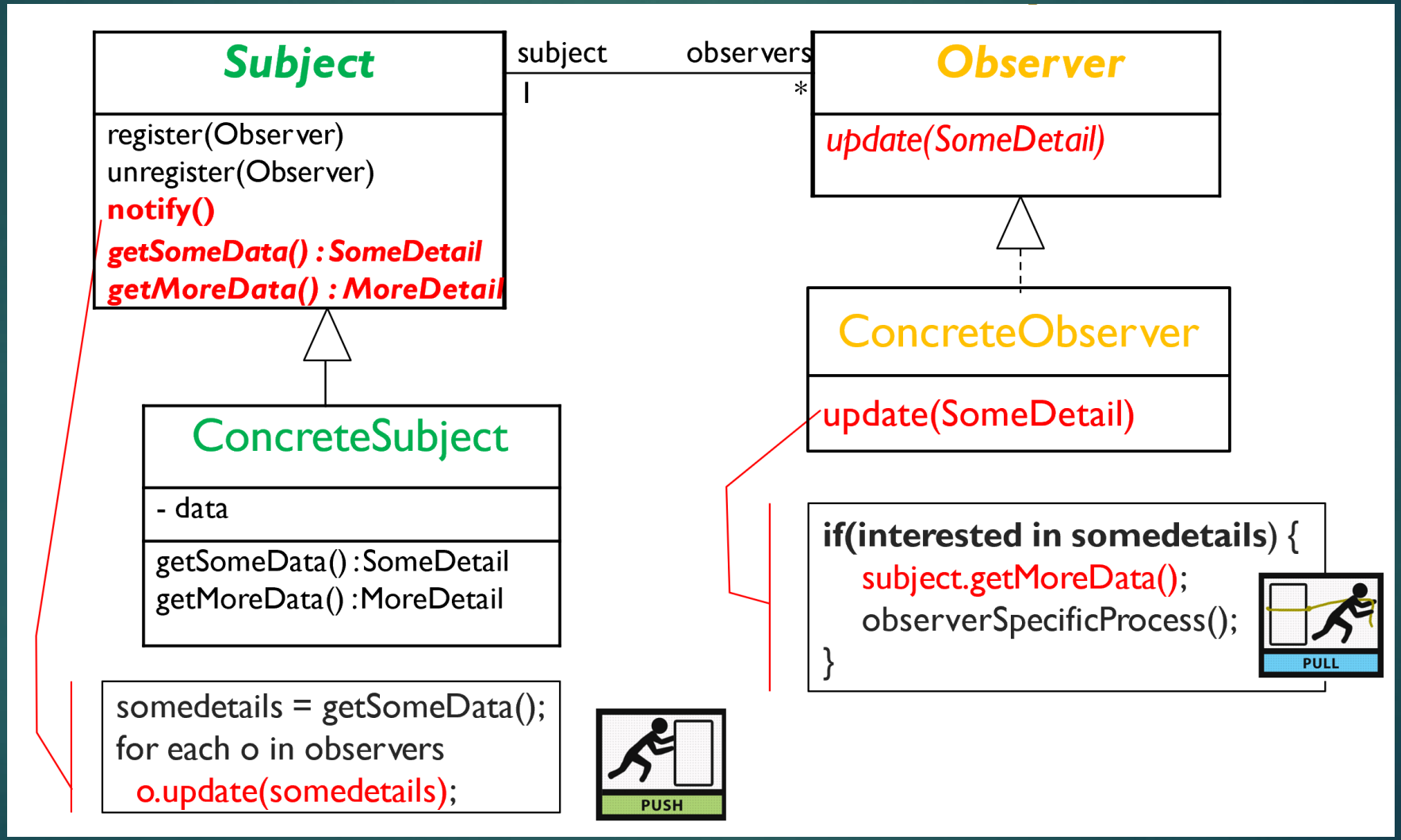
Push update

- Subject sends detailed information about the change (or update), whether Observer wants it (interested) or not – Observer does not need to call back – one way communication.



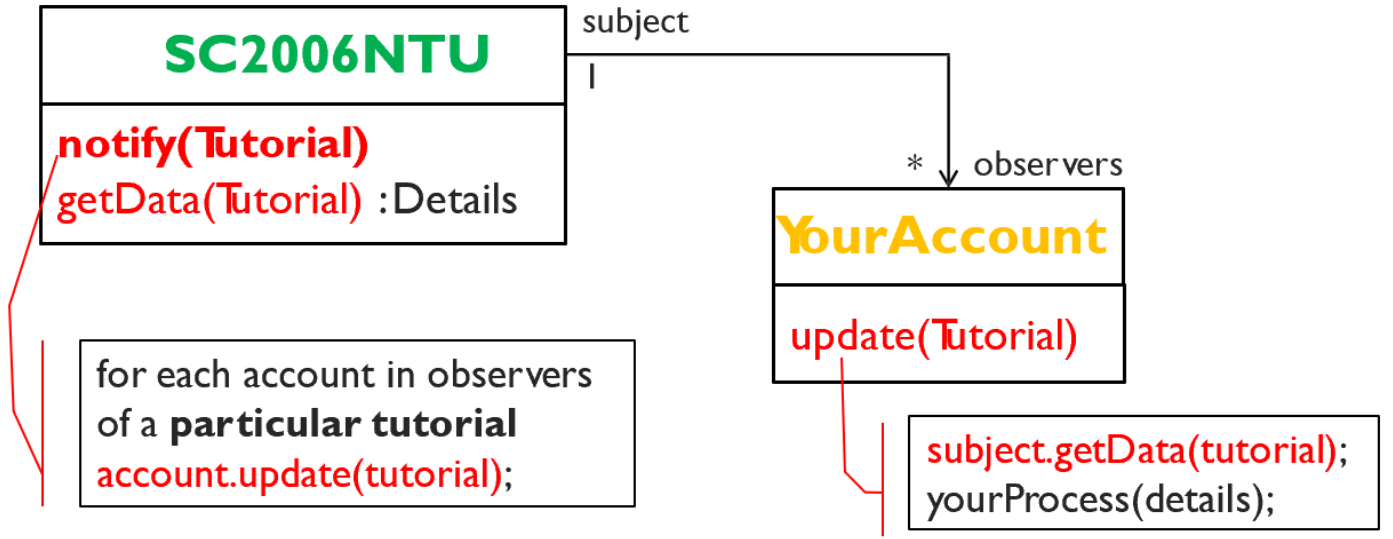


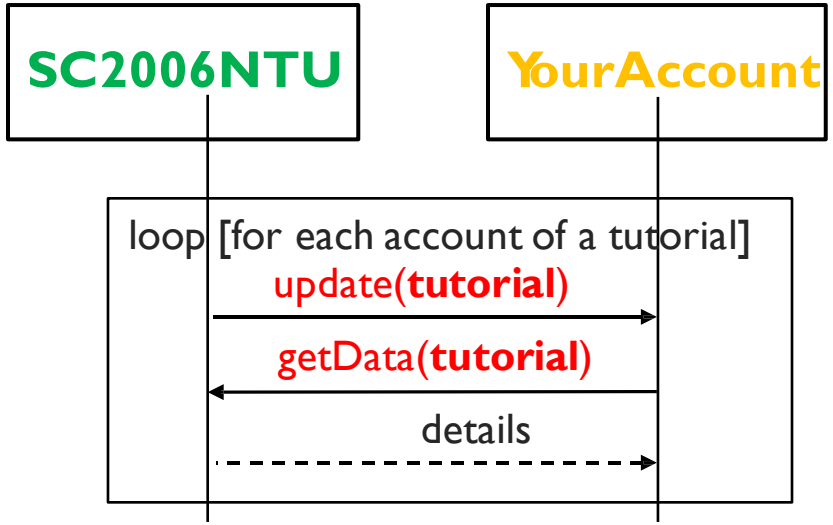
Observer Pattern – *Push + Pull Update*



Notification Mechanism – *Selective Notification*

- **SC2006NTU** notifies those who register a particular tutorial (i.e. interests)

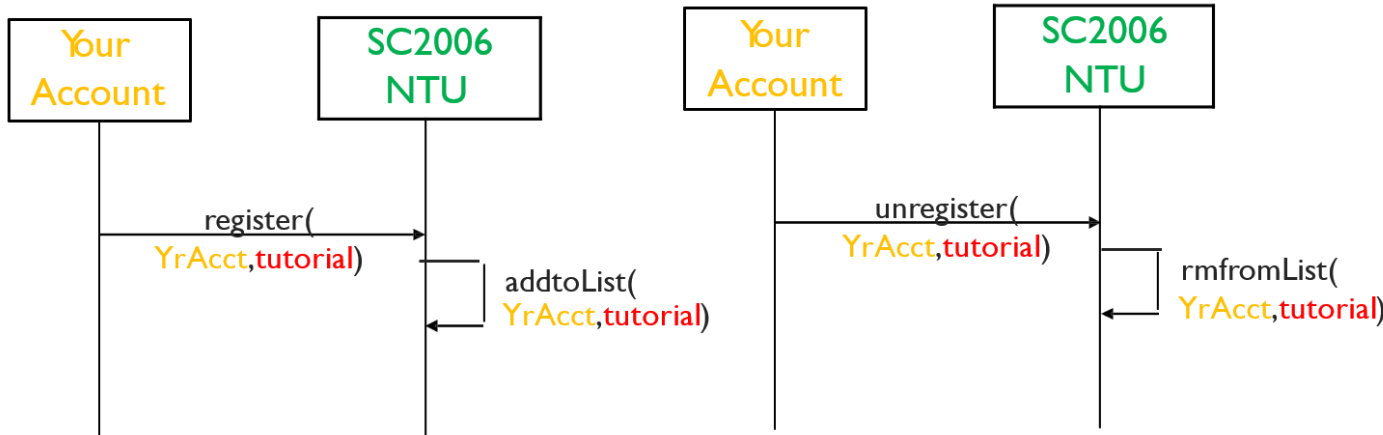




Subscription Mechanism – *Interests Subscription*

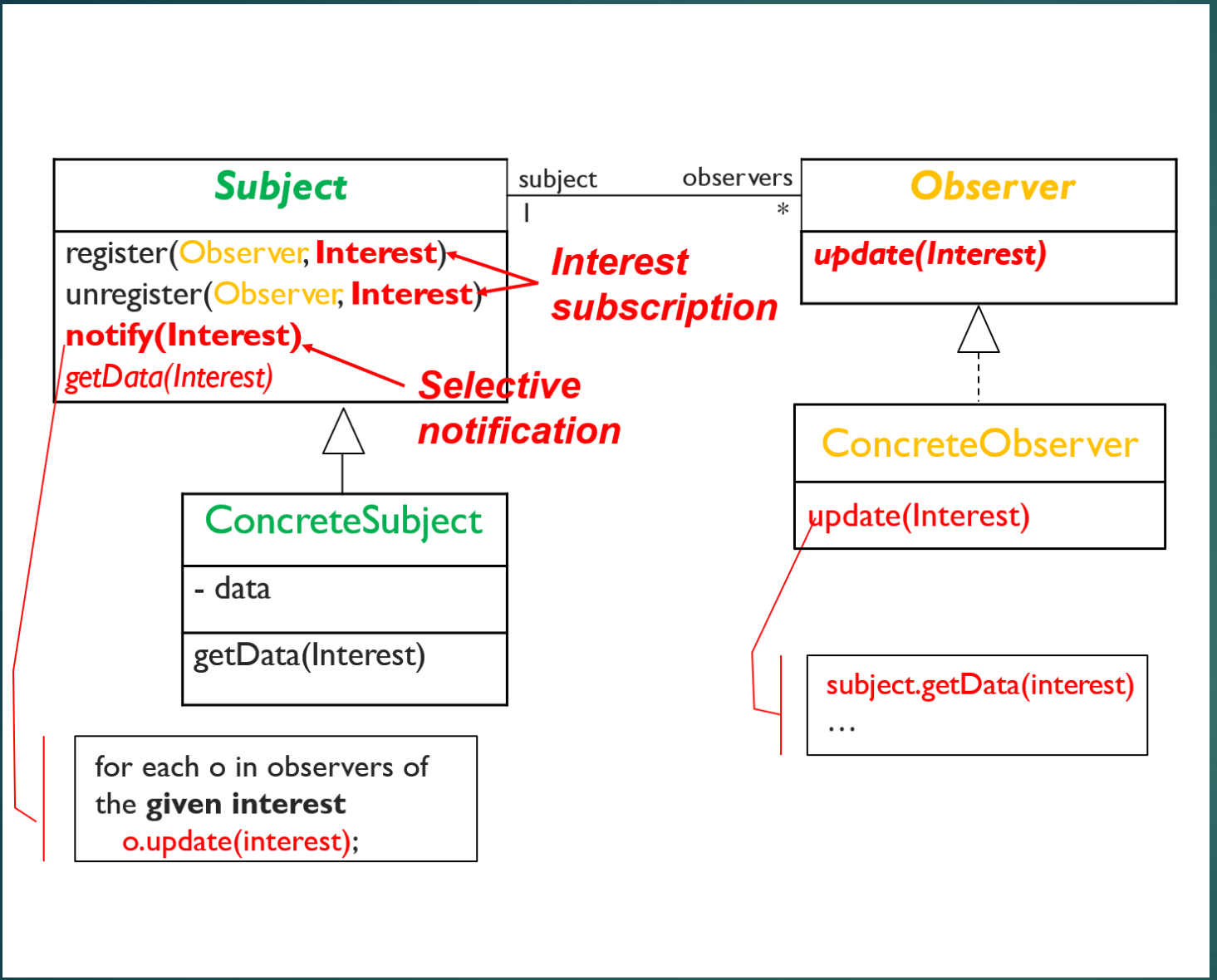
- Add to course (i.e., register)
- Remove from course (i.e., unregister)





*Indicate your **tutorial** session when registering the course*

Observer Pattern – Interest Subscription + Selective Notification



Event Handling – Design Problems

Loose coupling between an event source (subject) and its dependent event handlers (observers)

- Event source wants to notify event handlers the event once it occurs, but even handlers can change their interests in event source freely
- Event handlers want to know the evet once it occurs, but they do not know when the event will occur in event source

Event Handling & Observer Pattern – Design Problems

Event Handling

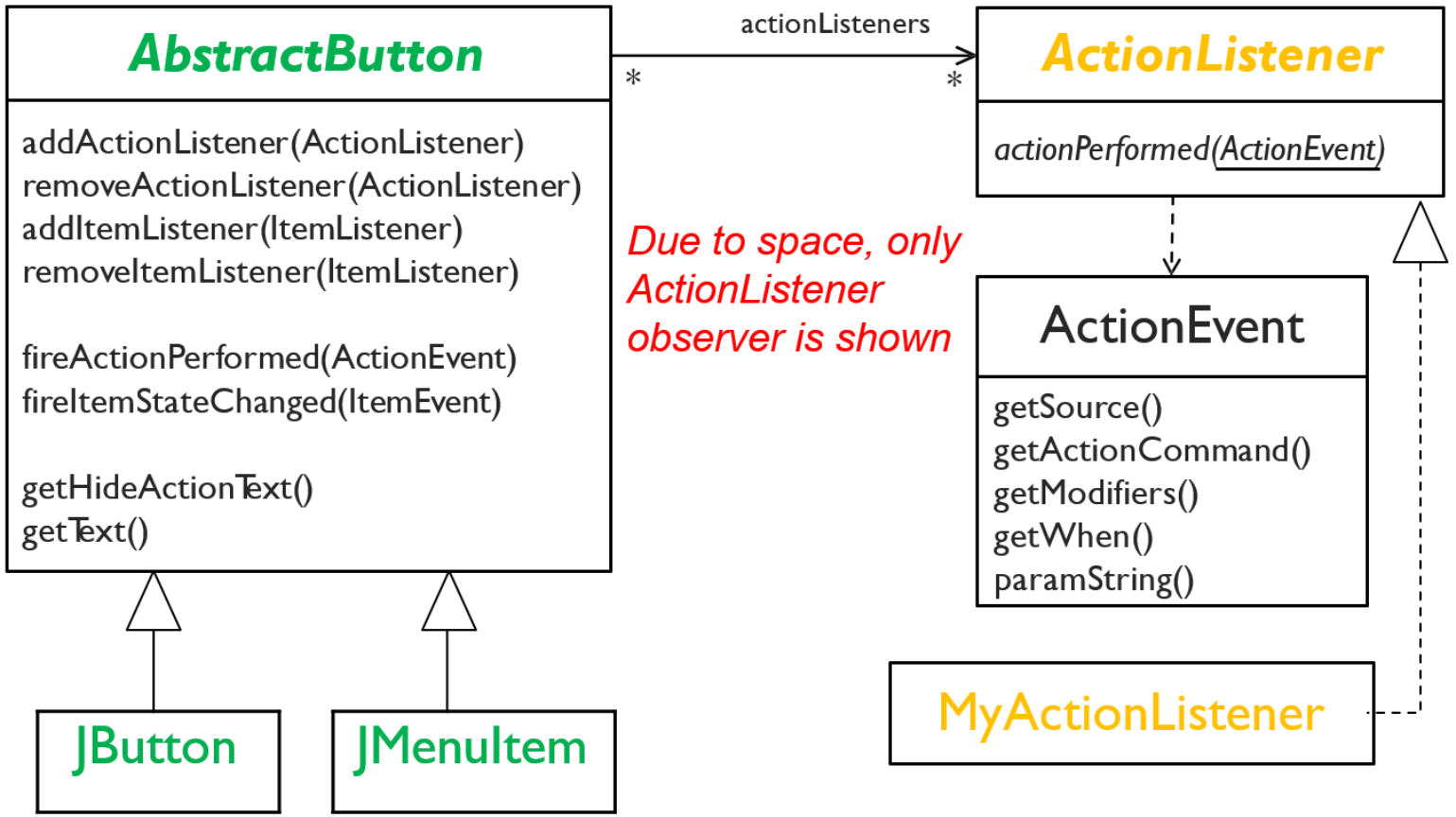
- Loose coupling between an event source (subject) and its dependent event handlers (observers)
  - Event source wants to notify event handlers the event once it occurs, but even handlers can change their interests in event source freely
  - Event handlers want to know the evet once it occurs, but they do not know when the event will occur in event source

Observer Pattern

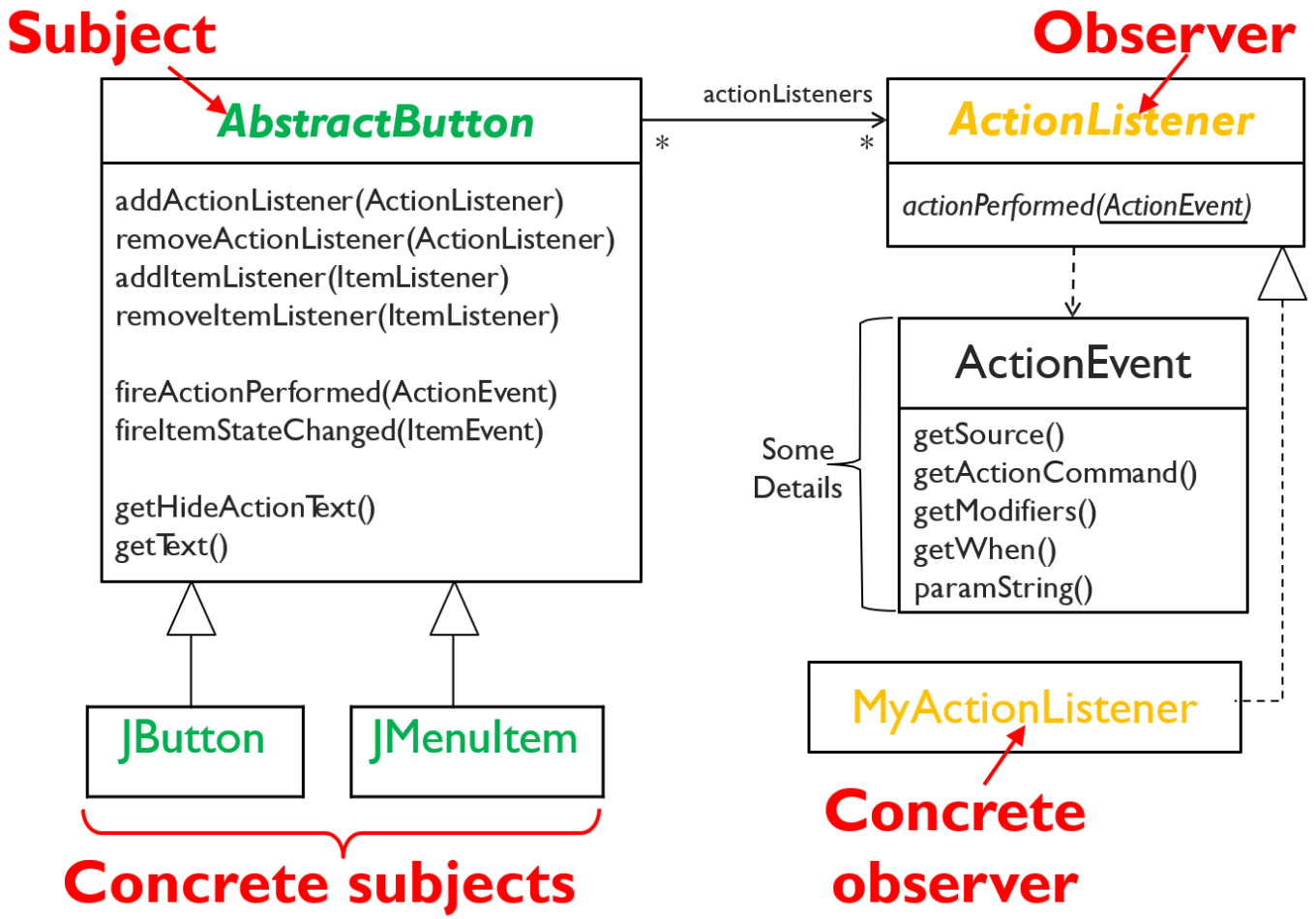
- Loose coupling between an object (subject) and its dependent objects (observers)
  - Subject wants to notify observers the change once it occurs, but observers can change their interests freely and constantly
  - Observers wants to know the change once it occurs, but they do not know when the change will occur

Java Swing Event Handling

- What are the event sources?
- What are the event handlers?



Java Swing Event Handling



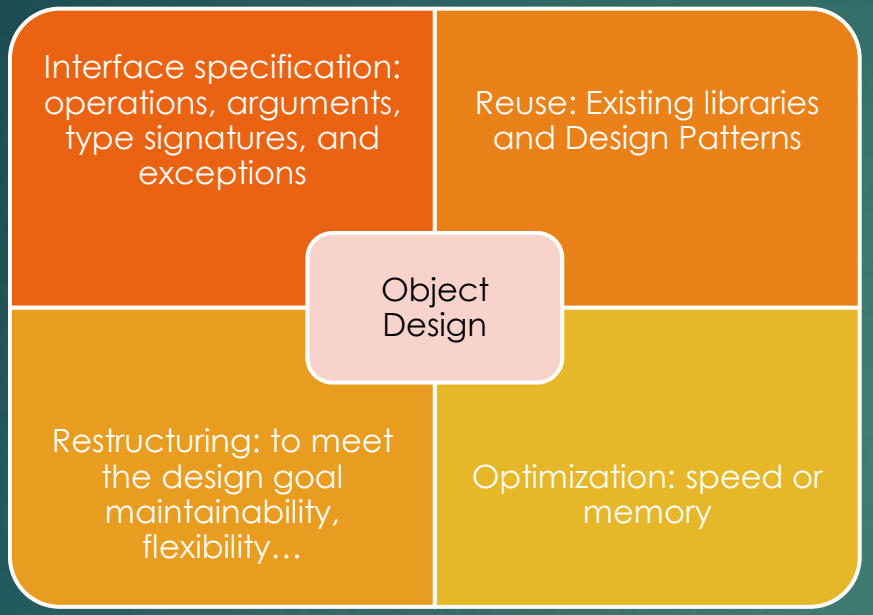
Observer

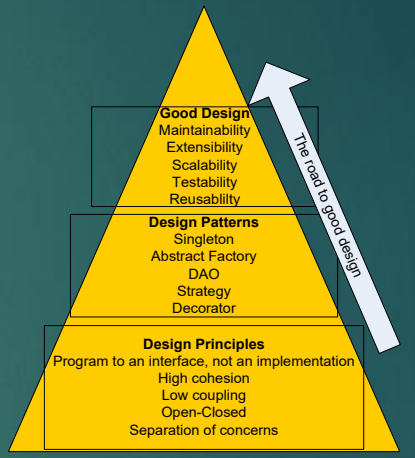
- Pros
  - Abstracts coupling between Subject and Observer
  - Supports broadcast communication
  - Enables reusability of subjects and observers independently of each other

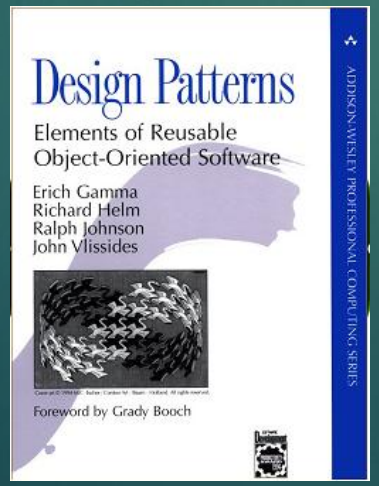
***Loose coupling** is a benefit for both sides!*

- Cons
  - Slower performance
  - If not used carefully the observer pattern can add unecessary complexity

Summary of Object Design







4 Elements of a Design Pattern

- Name
  - Describes the pattern
  - Adds to common terminology for facilitating communication (i.e. not just sentence enhancers)
- Problem
  - Describes when to apply the pattern
  - Answers - What is the pattern trying to solve?
- Solution
  - Describes elements, relationships, responsibilities, and collaborations which make up the design
- Consequences
  - Results of applying the pattern
  - Benefits and Costs
  - Subjective depending on concrete scenarios

8